1 Introduction

This text is an attempt to mathematically formalize Minecraft flying machines to ultimately prove that push limit 1 flying machines do not exist. The reader is supposed to be familiar with game mechanics and jargon. The game mechanics are formal enough as they are written in code. However, we will try to create a sufficiently simplified model of the game to make the task more feasible. For example, we make these assumptions right away:

- The world is infinite in all directions, unlike actual minecraft worlds
- In this text we do not account for any precision artifacts of float numbers and work with proper real numbers ($\mathbb{R}$)
- Random number generation (RNG) is non-deterministic and truly random

2 Crafting a formal definition for flying machines

Definition 1. A *structure* is a pair of sets (B, E), where B is a non-empty set of blocks with unique positions, and E is any set of entities. Here *blocks* and *entities* are treated as objects with position, state, and other data that would be provided by the game.

Definition 2. Let us define the *bounding box* of a structure. Denote by $\mathcal{P}$ the following set: $\left\{I_1 \times I_2 \times I_3 \mid I_j = (a, b) \cap \mathbb{Z};\ a, b \in \mathbb{Z} \cup \{\pm\infty\}\right\}$. For example, $\{0, 1, 2\} \times \{0, 1\} \times \mathbb{Z}$ belongs to $\mathcal{P}$.

The bounding box of a structure S is the smallest prism from $\mathcal{P}$ by inclusion, such that it contains S.

Definition 3. A structure is *finite* if it has a bounding box with finite dimensions. Otherwise, it is *infinite*.

Definition 4. We denote by *vacuum* a completely empty world with $doDayLightCycle = false$, $doWeatherCycle = false$, $randomTickSpeed = 0$, $doMobSpawning = false$, all chunks being force loaded. The rest of the settings are default. The game difficulty may vary depending on context.

Definition 5. Let S be a structure. Given $v \in \mathbb{Z}^3$, the notation $S + v$ means a new structure obtained from S by translating all objects in it by v. If $t \in \mathbb{N}_0 := \mathbb{N} \cup \{0\}$, we denote by $\mathcal{E}_t(S)$ (or just $\mathcal{E}_t S$) a new structure obtained by evaluating S in vacuum for t game ticks.

2.1 Clarifications on random events

In the beginning of the text I postulated true randomness, but let me elaborate what that actually means. Upon creating a vacuum world, we also generate a *truly random sequence* of integers (r_n). We then postulate that n-th call of "random" simply yields r_n. This effectively makes all game events deterministic; however, our sequence (r_n) generally lacks periodicity, unlike the pseudo random sequences that the actual game uses. This point of view is convenient for preventing certain structures from being formally periodic in time.

Proposition 1. *$\left\{\mathcal{E}_t \mid t \in \mathbb{N}_0\right\}$ endowed with composition operation is a commutative monoid. That is, the composition of evaluations is associative, $\mathcal{E}_0 = Id$, and*

$$\mathcal{E}_t \mathcal{E}_s = \mathcal{E}_s \mathcal{E}_t = \mathcal{E}_{t+s}.$$

2.2 Formal periodic flying machines

Definition 6. A *periodic flying machine* (or *PFM*) is any structure such that there exist $t \in \mathbb{N}, v \in \mathbb{Z}^3$, satisfying

$$\mathcal{E}_{t_0} S + v = \mathcal{E}_{t_0+t} S \quad \forall t_0 \in \mathbb{N}_0,\ \forall (r_n),$$

where the equality is to be understood as the equality of sets. Here (r_n) is the random sequence associated with the world.

Definition 7. A periodic flying machine is *stationary* if v can be chosen as $(0, 0, 0)$. Otherwise, it is *non-stationary* and such machines will be often referred to as *NPFM*.

Remark 1. We assume that objects in structures store enough data for the game to calculate the exact state of any structure after 1 tick using only that data and (r_n).

Remark 2. Our definition of a PFM includes all sorts of static structures that have nothing to do with slimestone. Even more concerning, it excludes many actual flying machines. For example, there exists a slime machine that moves along X axis between docking stations A and B, and each time it docks with B, the station increases the X coordinate by 1. However, this does not actually make the definition worthless. The aforementioned machine is simply a disjoint union of 3 periodic flying machines: 2 docking stations and one engine. The stations are stationary periodic flying machines with $t = 1, v = (0, 0, 0)$, which happen to move upon some perturbations from outside (in vacuum they do not happen, so formally the stations are indeed stationary). The engine can be viewed as an NPFM with $t = 10, v = (1, 0, 0)$.

Remark 3. Note that according to our definition, falling sand is not a periodic flying machine. The reason is that the state of a falling block includes a lifetime variable, which strictly increases with each game tick and hence $\mathcal{E}_t S \neq S + v$ for any $t \in \mathbb{N}, v \in \mathbb{Z}^3$. Lazy chunk shenanigans are not of any use, because we assume that all chunks are forceloaded.

At this point we could introduce "formal flying machines" as disjoint unions of periodic flying machines, but this is pointless because this class would include a ridiculous amount of possible structures, because a singular block placed in vacuum acts like a stationary periodic machine, unless it is a gravity block. In practice, all slime machines that are of any interest contain at least one NPFM. This reduces the question to investigating only NPFMs.

For now let us focus on analyzing finite NPFMs.

2.3 Choosing a feasible subclass of NPFMs

The simplest example of a finite NPFM is any finite set of vertical flowing water or lava blocks. However, this is rather boring. Conventionally, let us consider only NPFMs that do not use liquids.

For the same reason, I suggest disregarding entities from discussion, at least for now. That is because they add too much complexity to an already difficult task, not to mention that there are examples such as falling minecarts, fireballs, and wither projectiles.

3 General engines

Definition 8. Define E as the set of all finite NPFMs without liquids, entities, and command blocks. The letter E was chosen because it is effectively the set of engines.

Definition 9. Given $S \in E$, define the *period of* S as the smallest number $t \in \mathbb{N}$ for which there exists $v \in \mathbb{Z}^3 \setminus \{(0, 0, 0)\}$, such that $S + v = \mathcal{E}_t S$. We will use the following notation for periods: $P(S) := t$.

3.1 TODO

Show that in any structure from E, any movement of a block is caused by pistons pushing or pulling it.

Definition 10. Denote by E_1 the set E defined under the assumption that push limit is 1.

4 Reductions

Let us summarize what we have so far. Since we disregarded liquids, entities, and operator blocks like command blocks, the game can now be viewed as follows:

1. *Structures* are maps $S : \mathbb{Z}^3 \to \Sigma$, where Σ is the *alphabet* of in-game blocks, excluding blocks that we explicitly banned. For convenience, *air* is replaced with $\varnothing$.
2. We're only interested in structures with *finite support*, as they correspond to finite configurations of blocks.
3. $\mathcal{E}_1$ deterministically evaluates the next state of any structure according to some rule set matching the vanilla Minecraft with push limit 1.
4. Engines are defined as structures with finite support that satisfy $\mathcal{E}_T S = S + v$ for some $T \in \mathbb{N}$ and $v \in \mathbb{Z}^3 \setminus 0$.

We encode scheduled ticks into the alphabet, and the order of their execution into $\mathcal{E}$. Let's call this the *vanilla model*. The ultimate goal is to prove that engines do not exist. The idea is to create a simplified model of Minecraft that would satisfy these properties:

1. Any valid engine in the vanilla model must induce a valid engine in the simple model.
2. Σ is as small as possible.
3. The rule set is as small as possible
4. $\mathcal{E}$ of the simple model is allowed to be non-deterministic.

4.1 Example 1: Conscious pistons

This is a very stupid example, created mainly for presentation.

Let e_x, e_y, e_z be the basis vectors of $\mathbb{Z}^3$ and denote $D = \{\pm e_x, \pm e_y, \pm e_z\}$. Consider a simplified model with the following alphabet:

$$\Sigma = \{P_d : d \in D\} \cup \{\varnothing\}.$$

We treat this model as a game where we can make moves. Suppose we have a structure $S : \mathbb{Z}^3 \to \Sigma$, and $S(p) = P_d$ for some $p \in \mathbb{Z}^3,\ d \in D$. We will refer to P_d as to a "piston" facing towards d.

- If $S(p + d) = B \neq \varnothing$ and $S(p + 2d) = \varnothing$, then our piston can "push" B from $p + d$ to $p + 2d$.
- If $S(p + d) = \varnothing$ and $S(p + 2d) = B \neq \varnothing$, then the piston can "pull" B from $p + 2d$ to $p + d$.
- Otherwise there're no legal moves for that piston.

Obviously this makes this model non-deterministic, but in exchange we gain very low complexity, and any engine from the vanilla model trivially embeds into an engine in this model.

This model admits a very simple engine defined by the following structure:

$$S(0) = P_x, \quad S(e_x) = P_{-x}, \quad S(p) = \varnothing\ \forall p \neq 0, e_x.$$

4.2 Example 2: Conscious pistons with cargo

In the previous example, modify the alphabet to also include a "dead weight" block D. Define engines as before, but explicitly ask that engines must contain at least one D. Turns out that such engines also exist, so this model is not of any value.

4.3 Example 3: Game of Pistons 1 (GoP1)

Define

$$\Sigma = \{P_d : d \in D\} \cup \{\varnothing, R\},$$

where R represents a very simplified model of a power source. To describe how it interacts with pistons, define

$$A = D \cup (D + e_y) \setminus \{0\}.$$

In a structure S, a piston P_d at p is *active* if and only if there exists $q \in p+(A\setminus\{d\})$, such that $S(q) = R$. Active pistons can make moves according to the rule set of 4.1, by pushing and pulling blocks.

The activation rule adds a lot of complexity, and makes investigating this model highly difficult. Surprisingly, Claude AI managed to find an engine in this model in roughly 4 hours, see https://claude.ai/artifact/KAtdtahmKoTCsagzpbo1TP. It consists of 8 pistons and 4 source blocks. One cycle of that engine takes 278 moves to travel 1 block towards $+x$.

4.4 Example 4: Game of Pistons 2 (GoP2)

Recall that $D = \{\pm e_x, \pm e_y, \pm e_z\}$ and define

$$\Sigma = \{P_d^0, P_d^1 : d \in D\} \cup \{O_d : d \in D\} \cup \{\varnothing, H\}.$$

P_d^ε is a piston facing d, extended iif $\varepsilon = 1$, H stands for "hard-powerable block". O_f at q is an "observer" that outputs to $q-f$. Similarly to 4.3, we will have activation rules, but this time they're more restrictive. Let

$$A = D \cup (D + e_y) \setminus \{0\}.$$

Fix a structure $S : \mathbb{Z}^3 \to \Sigma$, and suppose $S(p) = P_d$ for some $p \in \mathbb{Z}^3$, $d \in D$. This piston is active if and only if one of the following conditions are met:

1. There exists $f \in D \setminus \{d\}$ such that $S(p + f) = O_f$.
2. There exists $f \in D \setminus \{-e_y\}$ such that $S(p + e_y + f) = O_f$.
3. There exist $q \in p + (A \setminus \{d\})$ and $f \in D$, such that $S(q) = H$ and $S(q + f) = O_f$.

Only active pistons may become extended. Define the set of movable states as

$$\mathrm{Mov} = \Sigma \setminus \{P_d^1 : d \in D\}.$$

Let $S(p) = P_d^\varepsilon$. Write $S[\mathrm{pos} \mapsto \text{new state}, \dots]$ to denote the structure obtained from S by modifying states at the listed positions.

(E1) If p is active, $\varepsilon = 0$ and $S(p + d) = \varnothing$: $S \rightsquigarrow S[p \mapsto P_d^1]$.

(E2) If p is active, $\varepsilon = 0$, $S(p+d) = B \in \mathrm{Mov}$ and $S(p+2d) = \varnothing$: $S \rightsquigarrow S[p \mapsto P_d^1, p+d \mapsto \varnothing, p+2d \mapsto B]$.

(R1) If $\varepsilon = 1$: $S \rightsquigarrow S[p \mapsto P_d^0]$.

(R2) If $\varepsilon = 1$, $S(p+d) = \varnothing$ and $S(p+2d) = B \in \mathrm{Mov}$: $S \rightsquigarrow S[p \mapsto P_d^0, p+d \mapsto B, p+2d \mapsto \varnothing]$.

Moves are made one at a time, in any order.

A careful reader experienced in Minecraft may notice that this rule set does not encode any abstraction of redstone blocks. Indeed, it doesn't. However, if non-existence of an engine in this model is proven, then necessarily all engines in the vanilla model must have redstone blocks.

After hours of bruteforcing with Claude and SAT solvers, a counter example was discovered: https://claude.ai/artifact/4N5vnEvoadpFVJUuDdv2vp. It uses 11 pistons, 1 solid block and 18 observers arranged into a "roof". The engine essentially pulls observers from the back of the sheet and transfers them to the front. One cycle takes 1388 elementary moves and translates the engine 1 block towards $+x$.

This hints that GoP3 must introduce block updates and shape updates to be restrictive enough. We no longer should admit anything resembling "conscious" pistons.